

Oryntra

Architecture and Design Specification

Live AI Product Review Room for Coding Agents

Purpose: This document is written for autonomous coding agents and human engineers. It defines the product architecture, review-room UX, IDE integration model, APIs, MCP surface, data models, security controls, and phased implementation plan.

Primary Name	Oryntra
Document Type	Architecture and Design Specification
Target Users	Developers, product owners, UX reviewers, AI coding agents
Primary Interfaces	Review Room Web App, local backend, IDE extension/MCP bridge, browser automation
Preferred Build Model	Separate browser-based review room connected to the IDE through a local backend
Version	1.0

1. Executive Summary

Oryntra is a local, IDE-connected product review environment for applications being built by AI coding agents. It lets a human open the running UI, click through real workflows, provide spoken or typed feedback, and have an agent correlate that feedback with browser state, DOM state, console/network evidence, and source code context.

The key product decision is to keep the main review experience in a separate browser-based Review Room while connecting it tightly back to Cursor, VS Code, and other IDEs through a local backend, MCP server, and optional IDE extensions. The IDE remains the code/diff/test surface; the Review Room becomes the product collaboration surface.

Architecture Principle: Do not cram the full experience into the IDE. Use the IDE as a launch, context, diff, and commit surface. Use the browser Review Room as the live collaboration surface.

2. Product Name Recommendation

Recommended name: Oryntra.

Why this name works:

- It clearly communicates guided review rather than generic code generation.
- It is broad enough to support UI, product behavior, requirements, accessibility, QA, and test review.
- It positions the product as a co-pilot for product review sessions, not just another IDE assistant.
- It can grow into enterprise workflows such as design review, demo review, security review, and release acceptance.

Name	Positioning	Pros	Concerns
Oryntra	AI-guided app review room	Clear, professional, broad	May overlap with generic review tools
PairUX	AI pair reviewer for UX behavior	Memorable and UI-focused	May sound UX-only
AgentReview	Agentic product review console	Direct and descriptive	Less brandable
LiveReview.dev	Developer-first live review room	Obvious domain-style name	May feel too narrow
SpecLoop	Turns feedback into specs and code	Strong workflow meaning	Less immediately obvious

3. Problem Statement

AI coding agents can write code, but they often lack the real-time human product intent that emerges while someone clicks through the actual UI. The missing loop is not just code review. It is live product behavior review: what the user expected, what the application actually did, and how the implementation should change.

3.1 Target Use Cases

- Review UI behavior while an AI coding agent is actively building or modifying an app.
- Convert spoken feedback into structured change requests and acceptance criteria.
- Link feedback to browser events, selected UI elements, screenshots, console logs, network calls, routes, and source files.
- Ask clarifying questions while the human is interacting with the live application.
- Generate and apply patches through Codex, Cursor, Claude Code, or another coding agent provider.
- Generate validation tests, especially Playwright end-to-end tests, based on the behavior the human expected.
- Keep product review, coding, diffs, and verification connected without overloading the IDE.

4. Architecture Overview



Primary Design Choice: The Review Room is a separate browser window at <http://localhost:4317/session/{sessionId}>. It is connected back to the IDE through the local backend and integration adapters.

4.1 Component Responsibility Matrix

Component	Responsibilities	Implementation Notes
Review Room Web App	Live app review, transcript, voice input, event timeline, change requests, approval controls	React/Vite or Next.js; WebSocket event stream; WebRTC later for realtime voice
Oryntra Backend	Session orchestration, event correlation, agent loop, persistence, API, security policy	Node.js/TypeScript with Fastify; localhost only by default
Browser Automation Service	Open app, capture DOM/accessibility snapshots, route changes,	Playwright and Playwright MCP

	console/network errors, screenshots	
IDE Bridge	Connect Review Room to current workspace, open files, show diffs, expose session context to IDE agents	Cursor MCP server first; VS Code extension next
Coding Agent Provider	Inspect code, propose patches, apply approved changes, generate tests	Provider abstraction for Codex, Cursor, Claude Code, local agents
Workspace Manager	Detect repo, create Git worktree, run allowed commands, manage test/build/lint	Git worktree per session for isolation
Persistence Layer	Store sessions, messages, events, change requests, patches, test results	SQLite for local MVP; Postgres optional later

5. Review Room Web App Design

The Review Room is the main collaboration surface. It should look and feel like a product review workspace rather than a code editor.

```
+-----+
| Session: clarion-ui | Branch: oryntra/abc123 | App: localhost:3000 | Agent |
+-----+
|                                     | AI Conversation / Transcript |
| Live App Preview / Controlled Browser | - human feedback           |
|                                     | - agent questions         |
| User clicks here, types here, reviews flows | - proposed changes       |
|                                     | - acceptance criteria     |
+-----+
| Event Timeline | Console | Network | DOM/Element | Change Requests | Tests |
+-----+
```

5.1 Review Room Panels

Panel	Purpose	MVP Requirements
Live App Preview	Surface the app being reviewed. The user clicks through workflows here.	Attach to app URL; show status and route; support screenshot capture.
AI Conversation	Human speaks or types feedback; agent responds with clarification and structured next steps.	Typed feedback first; push-to-talk in Phase 5.
Event Timeline	Show UI events linked to user feedback.	Click events, route changes, console errors, failed network requests.
Change Requests	Convert ambiguous feedback into buildable tasks.	Title, current behavior, expected behavior, acceptance criteria, linked evidence.
Console/Network	Expose debugging evidence without leaving the review room.	Console errors, failed requests, response codes.
Tests/Verification	Show whether the fix passed acceptance checks.	Manual in MVP, automated with Playwright in later phases.

6. IDE Integration Design

The Review Room must be connected back into the IDE, but the IDE should not host the full Review Room. The IDE should launch sessions, provide workspace context, receive file/diff/test events, and expose session data to coding agents.

```
IDE / Cursor / VS Code
  <-> Oryntra IDE Extension or MCP Bridge
    <-> Oryntra Backend
      <-> Review Room Web App
      <-> Browser Automation
      <-> Coding Agent Provider
      <-> Git Worktree / Tests
```

6.1 Cursor Integration

Cursor integration should use MCP first. Cursor documentation describes MCP as the mechanism for connecting Cursor to external tools and data sources. Oryntra should expose a local MCP server with session, browser, transcript, change request, and patch tools.

```
Cursor Agent asks Oryntra MCP:
- get_active_session
- get_browser_state
- get_recent_user_feedback
- get_review_transcript
- list_change_requests
- propose_patch
- run_verification
```

Cursor edits files in the attached workspace while Oryntra verifies behavior in the browser.

6.2 VS Code Integration

VS Code integration should be a lightweight extension. VS Code webviews are useful for custom views, but the full review room should remain external to avoid crowding the IDE.

- Command Palette: Oryntra: Start Session
- Command Palette: Oryntra: Attach Current Workspace
- Command Palette: Oryntra: Open Review Room
- Status bar: active session, app URL, branch/worktree
- Side panel: current change requests, transcript summary, latest verification status
- Open file requests: backend can ask the IDE to open a component or test file
- Diff view: IDE shows generated patches and lets the user approve or reject them

7. Core Data Models

7.1 Review Session

```
type ReviewSession = {
  id: string;
  workspacePath: string;
  repoName: string;
```

```

branchName: string;
worktreePath?: string;
appUrl: string;
devServerCommand?: string;
status: "starting" | "active" | "editing" | "testing" | "paused" | "closed";
ide?: "cursor" | "vscode" | "other";
createdAt: string;
updatedAt: string;
};

```

7.2 Browser Event

```

type BrowserEvent =
  | {
    id: string;
    sessionId: string;
    type: "click";
    timestamp: string;
    route: string;
    selector?: string;
    role?: string;
    text?: string;
    screenshotId?: string;
  }
  | {
    id: string;
    sessionId: string;
    type: "navigation" | "console_error" | "network_error" | "dom_snapshot";
    timestamp: string;
    route?: string;
    message?: string;
    url?: string;
    status?: number;
    payload?: unknown;
  };

```

7.3 Change Request

```

type ChangeRequest = {
  id: string;
  sessionId: string;
  title: string;
  userIntent: string;
  currentBehavior: string;
  expectedBehavior: string;
  affectedRoutes: string[];
  affectedComponents?: CodeReference[];
  priority: "low" | "medium" | "high";
  status:
    | "draft"
    | "needs-clarification"
    | "approved"
    | "in-progress"
    | "implemented"
    | "verified"
    | "rejected";
  acceptanceCriteria: string[];
  browserEvidence: BrowserEvent[];
  codeEvidence?: CodeReference[];
};

```

```
};
```

8. Local API Specification

8.1 Session Startup

```
POST /api/sessions
```

Request:

```
{
  "workspacePath": "/Users/steve/projects/clarion",
  "appUrl": "http://localhost:3000",
  "devCommand": "npm run dev",
  "ide": "cursor"
}
```

Response:

```
{
  "sessionId": "abc123",
  "reviewRoomUrl": "http://localhost:4317/session/abc123",
  "branch": "oryntra/abc123",
  "status": "active"
}
```

8.2 Browser State

```
GET /api/sessions/:id/browser-state
```

Response:

```
{
  "route": "/devices",
  "title": "Devices",
  "selectedElement": {
    "role": "button",
    "text": "View Details",
    "selector": "[data-testid='device-details-button']"
  },
  "consoleErrors": [],
  "networkErrors": [],
  "accessibilitySnapshotId": "snap_123"
}
```

8.3 Change Request Creation

```
POST /api/sessions/:id/change-requests
```

Request:

```
{
  "title": "Device detail should open in drawer",
  "currentBehavior": "Click navigates to a new page.",
  "expectedBehavior": "Click opens side drawer and preserves table state.",
  "acceptanceCriteria": [
    "Drawer opens on click",
    "Filters remain unchanged",
    "Scroll position is preserved"
  ]
}
```

```
}
```

9. MCP Server Tool Surface

Oryntra should expose an MCP server so Cursor and other MCP-capable coding agents can access review context without scraping logs or relying on manual copy/paste.

Tool	Purpose	Returns
start_review_session	Start a session for the active workspace	sessionId, reviewRoomUrl, branch
get_active_session	Return the current session	ReviewSession
get_browser_state	Return route, title, selected element, console/network state	BrowserState
get_recent_user_feedback	Return recent feedback linked to browser events	Voice/text utterances and evidence
get_review_transcript	Return full or summarized transcript	Transcript entries
create_change_request	Create a structured task from feedback	ChangeRequest
list_change_requests	List active tasks	ChangeRequest[]
propose_patch	Ask a provider to generate a patch	Patch summary and files changed
apply_patch	Apply an approved patch to the worktree	PatchResult
run_tests	Run configured verification commands	TestRunResult
verify_behavior	Use browser automation to validate acceptance criteria	VerificationResult
summarize_session	Create a handoff summary	Summary, decisions, open items

10. Browser Automation and Inspection

The browser automation layer should provide reliable UI state to the agent. Playwright MCP is a strong starting point because it exposes browser automation capabilities through MCP and structured accessibility snapshots, enabling LLMs to interact with web pages without depending only on screenshots.

- Launch a persistent Chromium context per session.
- Navigate to the configured app URL.
- Capture route changes, page title, console errors, failed network requests, and selected elements.
- Capture screenshots only when useful; prefer accessibility snapshots and DOM metadata for agent reasoning.
- Expose explicit test/verification commands instead of letting the agent randomly click around without state tracking.

11. Voice Design

Voice should be phased in after the review room and event correlation loop are working. Human voice input is the first priority; AI voice output can come later.

Phase	Voice Capability	Notes
-------	------------------	-------

MVP	Typed feedback only	Simplest path to validate the workflow.
Phase 5A	Push-to-talk human voice input	Speech-to-text creates transcript entries linked to current browser state.
Phase 5B	AI text response plus optional read-aloud	Lower complexity than full duplex.
Phase 5C	Realtime voice conversation	Use WebRTC voice-agent architecture; add barge-in and interruption handling.

12. Coding Agent Provider Abstraction

```
interface CodingAgentProvider {
  name: string;

  inspectCode(input: InspectCodeInput): Promise<CodeInspectionResult>;
  proposeChange(input: ChangeRequest): Promise<ProposedPatch>;
  applyPatch(input: ProposedPatch): Promise<PatchResult>;
  runTests(input: TestRunRequest): Promise<TestRunResult>;
  summarizeDiff(input: DiffSummaryRequest): Promise<string>;
}
```

12.1 Provider Strategy

- Do not hard-code Oryntra to one coding agent.
- Use a provider interface that can support Codex, Cursor, Claude Code, and local patch agents.
- Keep source edits isolated in a Git worktree until the user approves.
- Store all generated diffs and test results as session artifacts.
- Treat code edits as proposed changes until explicitly approved or configured otherwise.

13. Security and Safety Model

Security Posture: Oryntra touches source code, shell commands, browser state, local app sessions, and AI tooling. Default to localhost-only, workspace-only, approval-gated behavior.

Control	Requirement
Network binding	Bind backend to localhost only by default.
Workspace scope	Restrict file access to the workspace root and session worktree.
Shell commands	Use an allowlist. Require approval before running commands not in config.
Source modification	Use Git worktrees. Require approval before applying patches in MVP.
Secrets	Redact common secrets from logs, transcripts, screenshots, and MCP tool output.
Browser sessions	Use isolated browser contexts per session. Do not reuse user browser cookies by default.
MCP tools	Scope tools to active session and workspace. Avoid broad

	filesystem and shell tools.
Public sharing	Disable public tunnels unless explicitly enabled.

```

security:
  workspaceOnly: true
  localhostOnly: true
  requireApprovalBeforePatch: true
  requireApprovalBeforeShell: true
  redactSecrets: true
  allowPublicTunnel: false
  commandAllowlist:
    - npm run dev
    - npm run build
    - npm test
    - npm run test
    - npm run lint
    - pnpm dev
    - pnpm build
    - pnpm test
    - pnpm lint

```

14. Project Configuration

```

# oryntra.yaml

project:
  name: clarion-ui
  root: .

app:
  url: http://localhost:3000
  devCommand: npm run dev
  healthCheckUrl: http://localhost:3000/api/health

ide:
  preferred: cursor
  openDiffsInIde: true

agent:
  provider: codex
  requireApprovalBeforePatch: true
  requireApprovalBeforeShell: true

browser:
  headless: false
  captureScreenshots: true
  captureConsole: true
  captureNetwork: true

tests:
  testCommand: npm test
  lintCommand: npm run lint
  e2eCommand: npx playwright test

security:
  workspaceOnly: true
  redactSecrets: true
  allowPublicTunnel: false

```

15. Implementation Roadmap

Phase	Goal	Deliverables	Do Not Build Yet
1	Local Review Room	CLI, backend, session manager, React UI, attach to app URL, event timeline	Voice, code edits
2	Browser Observer	Playwright integration, console/network/route capture, screenshots, accessibility snapshots	Autonomous patching
3	Structured Feedback	Typed feedback to change requests, acceptance criteria, evidence linking	Voice output
4	Coding Agent Loop	Git worktree, provider abstraction, patch proposal, user approval, run tests	Full IDE UI replacement
5	Cursor + VS Code	MCP server, VS Code commands/status/side panel, open files/diffs	Complex custom IDE webview
6	Voice	Push-to-talk, STT, transcript linking, optional AI voice	Always-on multi-party meetings
7	Verification	Generated Playwright tests, before/after screenshots, acceptance checklist	Cloud collaboration

16. First Build Prompt for Coding Agents

Build the MVP for Oryntra, a local AI product review room for web apps.

Goal:

Create a local tool that lets a user attach to a running web app, open a browser-based review room, interact with the app, capture browser events, chat with an AI agent, and convert feedback into structured change requests.

Tech stack:

- TypeScript
- Node.js backend using Fastify
- React/Vite frontend
- Playwright for browser control
- SQLite for local session persistence
- WebSocket for live event streaming
- CLI command named oryntra

MVP requirements:

1. CLI command: `oryntra start --workspace . --url http://localhost:3000`
2. Start backend on `localhost:4317`.
3. Create a session ID.
4. Open review room at `/session/:id`.
5. Launch Chromium via Playwright and navigate to the provided app URL.
6. Capture current route, page title, click events, console errors, failed network requests, and screenshots on demand.
7. Show review room UI with app status, browser event timeline, chat panel, and change request panel.
8. Let user type feedback.

9. Convert feedback into structured change requests with title, current behavior, expected behavior, acceptance criteria, and linked browser events.
10. Store sessions, browser events, chat messages, and change requests in SQLite.
11. Do not edit source code yet.
12. Provide clean extension points for coding agent provider, MCP server, voice input, VS Code extension, and Cursor integration.

Security:

- Bind to localhost only.
- Do not run shell commands except starting the configured dev server.
- Do not modify source files in MVP.
- Restrict file access to workspace root.

Deliverables:

- Working repo
- README
- oryntra.yaml example
- npm scripts
- basic tests

17. MVP Acceptance Criteria

- A user can run `oryntra start --workspace . --url http://localhost:3000`.
- The backend starts on `localhost:4317` and creates a unique session.
- The Review Room opens in a browser and shows session status.
- A Playwright-controlled browser opens the target app URL.
- Click events, route changes, console errors, and failed network requests appear in the event timeline.
- The user can type feedback into the chat panel.
- The system can create a structured change request from feedback and selected browser evidence.
- All session artifacts persist in SQLite.
- The MVP does not modify source code.
- The codebase includes a clear provider abstraction for future coding-agent integration.

18. Reference Links

The following official documentation references informed the integration strategy:

- **Cursor MCP Documentation:** <https://cursor.com/docs/mcp>
- **VS Code Webview UX Guidelines:** <https://code.visualstudio.com/api/ux-guidelines/webviews>
- **VS Code Webview API Guide:** <https://code.visualstudio.com/api/extension-guides/webview>
- **Playwright MCP Documentation:** <https://playwright.dev/docs/getting-started-mcp>
- **Microsoft Playwright MCP Repository:** <https://github.com/microsoft/playwright-mcp>
- **OpenAI Realtime API Guide:** <https://developers.openai.com/api/docs/guides/realtime>
- **OpenAI Voice Agents Guide:** <https://developers.openai.com/api/docs/guides/voice-agents>